Topic: ScriptWell – Online Book Purchasing Store

ADVANCED INTERNET PROGRAMMING PROJECT REPORT

Submitted by:

Navneet Singh

UID: 26MCI10115

Submitted to:

Mr. Shivam Ojha

Submitted in partial fulfilment of the requirements for the degree of

MASTER OF COMPUTER APPLICATIONS (MCA) AIML

CHANDIGARH UNIVERSITY

University Institute of Computing

Punjab, India

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

BONAFIDE CERTIFICATE

This is to certify that the project report titled “ScriptWell – Online Book Purchasing Store” has been prepared as an academic project using Java Servlets, JavaServer Pages (JSP), JSTL, JDBC, HTML, CSS, JavaScript and Bootstrap, following the Model-View-Controller (MVC) architecture on Apache Tomcat 10.1 with JDK 17. The project demonstrates catalogue browsing, search and filters, user registration and login, session management, a shopping cart with AJAX updates, Buy Now, checkout with simulated payment, order history and an administrator dashboard for book, inventory and order management, backed by an Oracle database.

This project report is submitted in partial fulfilment of the requirements for the degree of Master of Computer Applications (MCA) AIML.

Student Signature: _____ Project Supervisor: _____

Head of Department: _____ Date: _____

TABLE OF CONTENTS

Chapter 1: Introduction and Background Study .....PAGE 1 - 2

- 1.1 Project Overview
- 1.2 Problem Definition
- 1.3 Objectives and Scope
- 1.4 Technologies Used
- 1.5 Existing System and Proposed Solution

Chapter 2: System Requirements and Design .....PAGE 3 - 4

- 2.1 Hardware and Software Requirements
- 2.2 System Architecture (MVC)
- 2.3 Request Flow and Program Workflow
- 2.4 Project Structure and Modules
- 2.5 Functional and Non-Functional Requirements

Chapter 3: Implementation.....PAGE 5 - 7

- 3.1 Servlet Controllers and URL Mapping
- 3.2 JSP Views, JSTL and Custom Tag
- 3.3 Database Access with JDBC CallableStatement
- 3.4 Session Management, Filters and Security
- 3.5 Client-side Scripting and AJAX
- 3.6 User Interface and Responsive Design

Chapter 4: Testing, Results and Conclusion .....PAGE 8 - 12

- 4.1 Testing Methodology
- 4.2 Test Cases and Validation
- 4.3 Results and Screenshots
- 4.4 Limitations and Future Scope
- 4.5 Conclusion
- 4.6 Pushing the Project to GitHub
- References

CHAPTER 1:

INTRODUCTION AND BACKGROUND STUDY

1.1 Project Overview

ScriptWell is a modern, responsive online bookstore built as a dynamic Java web application. Visitors can browse 34 books in seven categories, search by title, author or ISBN, filter by category and price, and open detailed book pages. Registered customers can manage a shopping cart, buy a single book instantly with **Buy Now**, check out with a simulated Card, UPI or Cash-on-Delivery payment, see an order confirmation and follow their order history. An administrator gets a dashboard with sales figures and full control over books, stock and orders.

This report covers the **web tier** of the project. The application is written with Java Servlets and JSP and follows the **Model-View-Controller (MVC)** pattern. It runs on Apache Tomcat 10.1 with JDK 17 and is developed in Eclipse IDE. The data is stored in Oracle Database Free, which the web tier reaches only through JDBC `CallableStatement` calls to stored PL/SQL.

1.2 Problem Definition

A small bookstore needs an online channel that is easy to use on any device, secure, and simple for staff to manage. Static web pages cannot do this, because the catalogue, cart, stock and orders change all the time. The specific problems to solve were:

- Generate pages dynamically from database data while keeping presentation code separate from business logic.
- Remember who the user is and what is in their cart across many HTTP requests, which are stateless.
- Protect customer pages and the administrator area from unauthorised access.
- Give a fast, app-like experience (for example, adding to the cart without reloading the page) on desktop and mobile.

1.3 Objectives and Scope

Objectives

1. Build a dynamic web application with **Java Servlets** as controllers and **JSP + JSTL** as views, following **MVC**.
2. Use **JDBC** with **CallableStatement** for all database access through a DAO layer.
3. Implement **session management**, **servlet filters** for authentication and role-based authorisation, and the Post/Redirect/Get pattern.
4. Design a clean, premium and **responsive** interface with **HTML5, CSS3, Bootstrap 5** and **JavaScript**.
5. Provide search, filters, book details, registration/login, cart, Buy Now, checkout, confirmation and order history.
6. Provide an administrator dashboard for book CRUD, inventory and order management.

Scope

The application supports guests, customers and administrators. Payment is simulated: card numbers are checked for format with the Luhn algorithm, but no money is charged and no card data is stored. E-mail notifications, reviews and a real payment gateway are outside the scope and are listed as future work.

1.4 Technologies Used

Table 1.1: Technologies and tools

Technology	Purpose in the project
Java Servlets (Jakarta Servlet 6.0)	Controllers that handle HTTP GET/POST requests
JSP 3.1, EL and JSTL 3.0	Server-side views without Java scriptlets; core, fmt and functions tags
JDBC (ojdbc11) and CallableStatement	Calling stored PL/SQL procedures and functions in Oracle
HTML5, CSS3, Bootstrap 5.3, Bootstrap Icons	Page structure, custom styling and responsive grid
JavaScript (Fetch API) and Chart.js	AJAX add-to-cart, form helpers and the admin sales chart
Apache Tomcat 10.1, JDK 17	Servlet container and Java runtime
Eclipse IDE (Dynamic Web Project)	Development, building and deployment to Tomcat
Oracle Database Free 23ai	Data storage and business logic in PL/SQL

1.5 Existing System and Proposed Solution

Table 1.2: Comparison of the existing approach and ScriptWell

Aspect	Typical existing system	Proposed ScriptWell application
Pages	Static HTML or scriptlet-heavy JSP	MVC: servlets prepare data, JSTL views render it
User state	None or cookies only	HttpSession with session-fixation protection
Access control	Checks copied into every page	One AuthFilter for all protected URLs
Cart updates	Full page reload	AJAX with the Fetch API and JSON responses
Data access	SQL strings in JSP pages	DAO layer using CallableStatement only
Devices	Desktop-only layout	Responsive Bootstrap layout for phones and desktops

CHAPTER 2:

SYSTEM REQUIREMENTS AND DESIGN

2.1 Hardware and Software Requirements

Table 2.1: Hardware and software requirements

Item	Requirement
Processor / Memory	64-bit dual-core CPU or better, minimum 8 GB RAM
Operating system	Windows 10/11, Linux or macOS
Java	JDK 17 (the code is compiled with --release 17)
Web server	Apache Tomcat 10.1
IDE	Eclipse IDE for Enterprise Java and Web Developers
Database	Oracle Database Free 23ai (FREEPDB1)
Libraries	ojdbc11, Jakarta JSTL 3.0 (in WEB-INF/lib); Bootstrap 5.3 and Chart.js from CDN
Client	Any current browser: Chrome, Edge or Firefox

2.2 System Architecture (MVC)

The application has a three-tier architecture, and the middle tier is organised as MVC (Figure 2.1). The **Model** is a set of JavaBeans (`Book`, `Category`, `User`, `CartItem`, `Order`). The **View** is JSP pages stored under `WEB-INF/views`, so the browser can only reach them through a controller. The **Controller** layer is seven servlets that read the request, call the DAO layer and choose the view. Two filters handle cross-cutting concerns before any servlet runs.

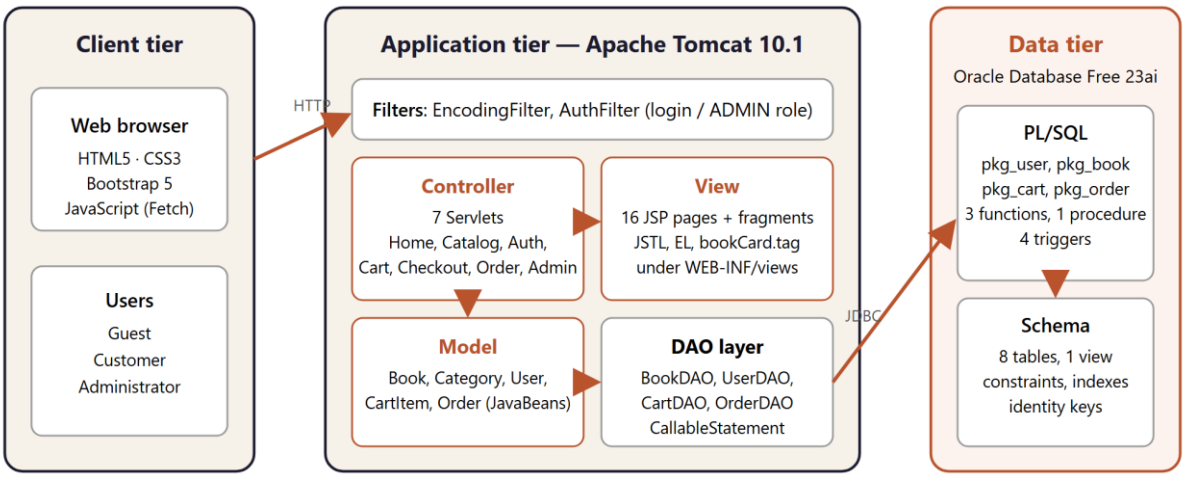


Figure 2.1: Three-tier MVC architecture of ScriptWell

2.3 Request Flow and Program Workflow

The workflow for placing an order shows how the parts work together:

1. The browser submits the checkout form with `POST /checkout`.
2. `EncodingFilter` sets UTF-8. `AuthFilter` checks that the session contains a signed-in user and otherwise redirects to `/login?next=/checkout`.

3. `CheckoutServlet` reads the address fields and validates the simulated payment (card format and Luhn check, UPI pattern, or COD).
4. `OrderDAO.placeOrder()` calls the stored procedure `pkg_order.place_order` through a `CallableStatement` and receives the order ID.
5. The servlet updates the cart badge count in the session and sends a redirect to `/order?id=...&placed=1` (Post/Redirect/Get).
6. `OrderServlet` loads the order and forwards to `order.jsp`, which shows the confirmation page.

2.4 Project Structure and Modules

```
scriptwell/
|-- src/main/java/
|   |-- db.properties                JDBC URL, user and password
|   `-- com/scriptwell/
|       |-- controller/   HomeServlet, CatalogServlet, AuthServlet, CartServlet,
|       |                 CheckoutServlet, OrderServlet, AdminServlet
|       |-- dao/          BookDAO, UserDAO, CartDAO, OrderDAO
|       |-- model/        Book, Category, User, CartItem, Order
|       |-- filter/       AuthFilter, EncodingFilter
|       `-- util/         DBConnection, WebUtil
|-- src/main/webapp/
|   |-- WEB-INF/web.xml, views/*.jsp, views/admin/*.jsp, tags/bookCard.tag, lib/
|   `-- assets/           css/style.css, js/app.js, img/, covers/ (35 SVG covers)
`-- database/             Oracle schema, PL/SQL packages, triggers, seed data
```

Listing 2.1: Eclipse Dynamic Web Project structure

Table 2.2: Application modules

Module	Main features
Catalogue	Home page (top rated, new arrivals, categories), browse page with search, filters and sorting, book details
Accounts	Registration with validation, login, logout, safe redirect back to the requested page
Cart and checkout	AJAX add-to-cart, quantity update, remove, clear, Buy Now, simulated payment, confirmation
Orders	Order history, order details with status timeline, cancellation before shipping
Administration	Dashboard and chart, book CRUD, inventory and restock, order status, customer list

2.5 Functional and Non-Functional Requirements

Functional requirements

- Display, search, filter and sort the book catalogue, and show book details with related titles.
- Register and authenticate users. Keep the user signed in for the session.
- Add, update, remove and clear cart items. Support Buy Now and checkout with Card, UPI or COD.
- Show order confirmation and history, and let administrators manage books, stock and orders.

Non-functional requirements

- **Responsiveness:** layouts adapt from 360 px phones to wide desktop screens.
- **Security:** filters for authentication and authorisation, output escaping, HttpOnly session cookie.
- **Maintainability:** strict MVC layering, one DAO per area, reusable JSP fragments and tag file.

CHAPTER 3:

IMPLEMENTATION

3.1 Servlet Controllers and URL Mapping

The servlets are registered with the `@WebServlet` annotation instead of XML. Each one handles GET requests by loading data and forwarding to a view. POST requests change data and end with a redirect, so refreshing the browser never resubmits a form.

Table 3.1: Servlets and URL patterns

Servlet	URL pattern(s)	Responsibility
HomeServlet	"", /home	Categories, top-rated books and new arrivals
CatalogServlet	/books, /book	Search, filter and sort listing; book details with related books
AuthServlet	/login, /register, /logout	Sign in, create account, sign out
CartServlet	/cart	Show cart; POST add, update, remove, clear; JSON reply for AJAX
CheckoutServlet	/checkout	Cart or Buy Now summary, payment validation, order placement
OrderServlet	/orders, /order	Order history, details and confirmation, cancellation
AdminServlet	/admin/*	Dashboard, books CRUD, inventory, orders, customers

```
@WebServlet(urlPatterns = { "", "/home" })
public class HomeServlet extends HttpServlet {
    private final BookDAO bookDAO = new BookDAO();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        try {
            req.setAttribute("categories", bookDAO.categories());
            req.setAttribute("featured", bookDAO.featured(8));
            req.setAttribute("arrivals", bookDAO.newArrivals(6));
        } catch (SQLException e) {
            req.setAttribute("error", DBConnection.friendlyMessage(e));
        }
        req.getRequestDispatcher("/WEB-INF/views/home.jsp").forward(req, resp);
    }
}
```

Listing 3.1: HomeServlet - a controller that prepares data and forwards to a view

3.2 JSP Views, JSTL and Custom Tag

The views use Expression Language and the JSTL `core`, `fmt` and `functions` libraries. They contain no Java scriptlets. Shared markup (navigation bar, flash messages, footer and scripts) lives in `common/header.jspf` and `common/footer.jspf`. The book card, used on the home, catalogue and related-books sections, is a **JSP tag file**, so its markup is written only once:

```
<%@ attribute name="book" required="true" type="com.scriptwell.model.Book" %>
...
<c:forEach items="${featured}" var="b">
  <div class="col-6 col-md-4 col-lg-3"><sw:bookCard book="${b}" /></div>
</c:forEach>

<span class="price">&#8377;<fmt:formatNumber value="${book.price}" maxFractionDigits="0"/></span>
<a href="${ctx}/book?id=${book.id}">${fn:escapeXml(book.title)}</a>
```

Listing 3.2: Using the bookCard tag file, fmt and fn:escapeXml

A context parameter in `web.xml` sets the JSTL locale to `en_IN`, so rupee amounts use Indian digit grouping. Custom error pages for HTTP 403, 404 and 500 are also configured in `web.xml`.

3.3 Database Access with JDBC CallableStatement

Every DAO method opens a connection in a try-with-resources block, prepares a JDBC escape call to a stored procedure, binds the IN parameters, registers the OUT parameters and executes the call. Query results come back as a REF CURSOR, which is read as a normal `ResultSet` and mapped to model objects:

```
public List<Book> search(String keyword, Integer categoryId, Double minPrice,
                         Double maxPrice, String sort) throws SQLException {
    try (Connection con = DBConnection.getConnection();
         CallableStatement cs = con.prepareCall("{call pkg_book.search_books(?,?,?,?,?,?)}")) {
        setString(cs, 1, keyword);            // setNull(...) when a filter is empty
        setInt(cs, 2, categoryId);
        setDouble(cs, 3, minPrice);
        setDouble(cs, 4, maxPrice);
        setString(cs, 5, sort);
        cs.registerOutParameter(6, Types.REF_CURSOR);
        cs.execute();
        return readBooks(cs.getObject(6, ResultSet.class));
    }
}
```

Listing 3.3: BookDAO.search() calling a stored procedure

Stored functions use the `{? = call ...}` syntax, for example `{? = call pkg_cart.cart_total(?)}`. `DBConnection` loads its settings from `db.properties`. Its `friendlyMessage()` method turns Oracle application errors (codes -20000 to -20999) into messages for the user, and logs any other error.

3.4 Session Management, Filters and Security

After a successful login, the old session is invalidated and a new one is created. This prevents **session fixation**. The session stores the `User` bean and the cart count shown on the navigation badge. One-time **flash messages** are kept in the session and removed after they are displayed. `AuthFilter` protects all customer and administrator URLs:

```
@WebFilter(urlPatterns = { "/cart", "/checkout", "/orders", "/order", "/admin/*" })
public class AuthFilter implements Filter {
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse resp = (HttpServletResponse) response;
        User user = WebUtil.currentUser(req);
        if (user == null) {                              // not signed in
            resp.sendRedirect(req.getContextPath() + "/login?next=" + URLEncoder.encode(
                    req.getServletPath(), StandardCharsets.UTF_8));
            return;
        }
        if (req.getServletPath().startsWith("/admin") && !user.isAdmin()) {
            resp.sendError(HttpServletResponse.SC_FORBIDDEN, "Administrators only");
            return;
        }
        chain.doFilter(request, response);
    }
}
```

Listing 3.4: AuthFilter (simplified)

- All dynamic text in the views is escaped with `fn:escapeXml` to prevent cross-site scripting.

- All database calls use bound parameters, so SQL injection is not possible.
- The session cookie is `HttpOnly` (set in web.xml). The `next` redirect only accepts paths inside the application.
- Passwords are hashed with SHA-256 in the database and are never stored or logged in plain text.

3.5 Client-side Scripting and AJAX

`assets/js/app.js` adds behaviour on top of the pages:

- **AJAX add-to-cart:** the form is sent with `fetch()` and the header `X-Requested-With: fetch`. `CartServlet` replies with JSON, and the script updates the cart badge and shows a toast without reloading the page.
- **Quantity steppers** on the book and cart pages. Cart rows submit automatically when the quantity changes.
- **Checkout helpers:** shows only the fields for the chosen payment method, formats the card number and expiry, uses HTML5 validation, and shows a short "Processing payment" state.
- **Admin:** confirmation dialogs before deletes, and a Chart.js bar chart of revenue by category.

```
const res = await fetch(form.action, {
  method: "POST",
  headers: { "X-Requested-With": "fetch" },
  body: new URLSearchParams(new FormData(form)),
});
const data = await res.json();          // {"ok":true,"count":3,"message":"Added to your cart"}
toast(data.message, data.ok);
document.getElementById("cartBadge").textContent = data.count;
```

Listing 3.5: AJAX add-to-cart with the Fetch API

3.6 User Interface and Responsive Design

The interface is built on the Bootstrap 5 grid with a custom stylesheet that defines design tokens: a deep ink colour, a terracotta accent, gold highlights, rounded cards and soft shadows. Headings use the *Playfair Display* font and body text uses *Inter*. Each of the 35 book covers is a generated SVG with a palette for its category. The grid changes from four columns on desktop to two on phones, and the admin sidebar becomes a horizontal scrolling menu on small screens.

CHAPTER 4:

TESTING, RESULTS AND CONCLUSION

4.1 Testing Methodology

The application was deployed on Apache Tomcat 10.1 and tested with **black-box functional testing**. Every servlet was exercised with valid and invalid input using real HTTP requests, including redirects, status codes and JSON responses. Then complete user journeys for customers and administrators were tested in the browser at desktop and mobile widths. Test inputs were chosen with equivalence partitioning and boundary values (for example quantity 1, the available stock, and stock + 1).

4.2 Test Cases and Validation

Table 4.1: Web application test cases

No.	Test case	Expected result	Status
1	Open home, catalogue and book pages	HTTP 200; 8 top-rated books on home, 34 in catalogue	Pass
2	Search 'java'; category + price filters	2 results; 6 results; 16 results in range	Pass
3	Open /book?id=999	Custom 404 error page	Pass
4	Register with an existing e-mail	Error message, form kept with entered values	Pass
5	Login with wrong / correct password	Error message / redirect, admin goes to dashboard	Pass
6	Open /cart without logging in	302 redirect to /login?next=/cart	Pass
7	AJAX add to cart	JSON {ok:true,count:1}; badge and toast updated	Pass
8	Add more copies than in stock	“Only 1 copy ... available”, cart unchanged	Pass
9	Checkout with an invalid card number	Luhn check fails, form shown again with error	Pass
10	Checkout with valid test card / Buy Now with UPI	Order created, redirect to confirmation page	Pass
11	Customer opens another user's order	HTTP 404	Pass
12	Customer opens /admin	HTTP 403 from AuthFilter	Pass
13	Admin adds, edits and deletes a book	Changes saved; flash message shown	Pass
14	Admin updates order status	Status badge changes; history kept	Pass

4.3 Results and Screenshots

All test cases passed. The screenshots below were taken from the running application.

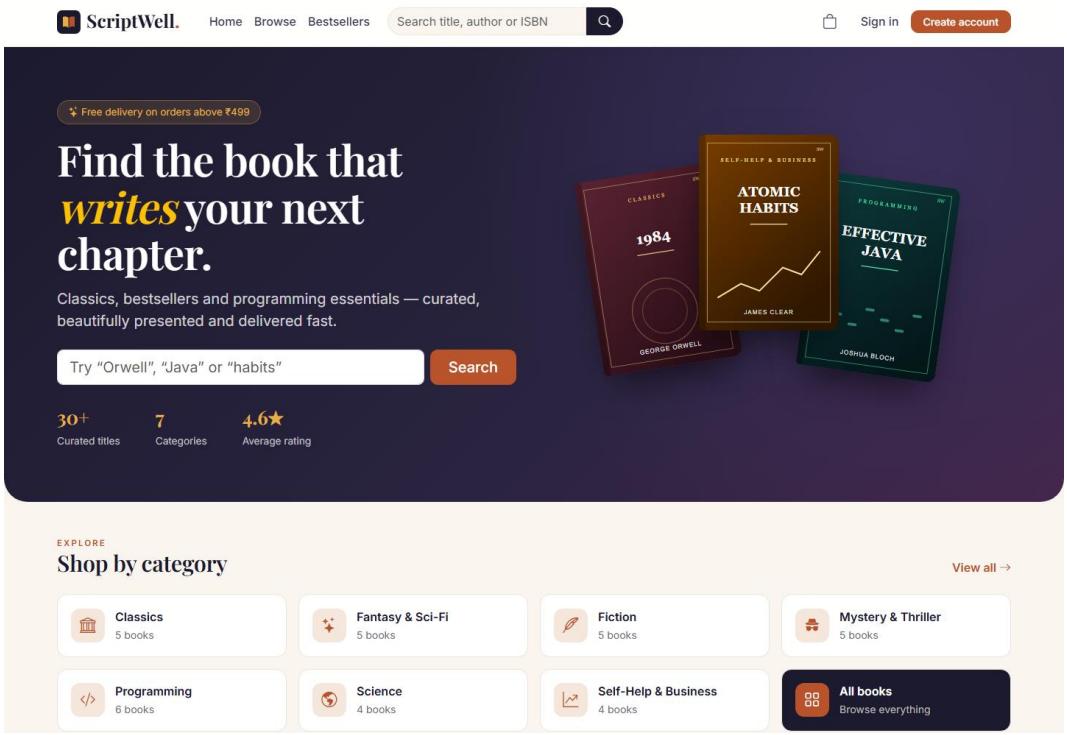


Figure 4.1: Home page with hero section, search and categories

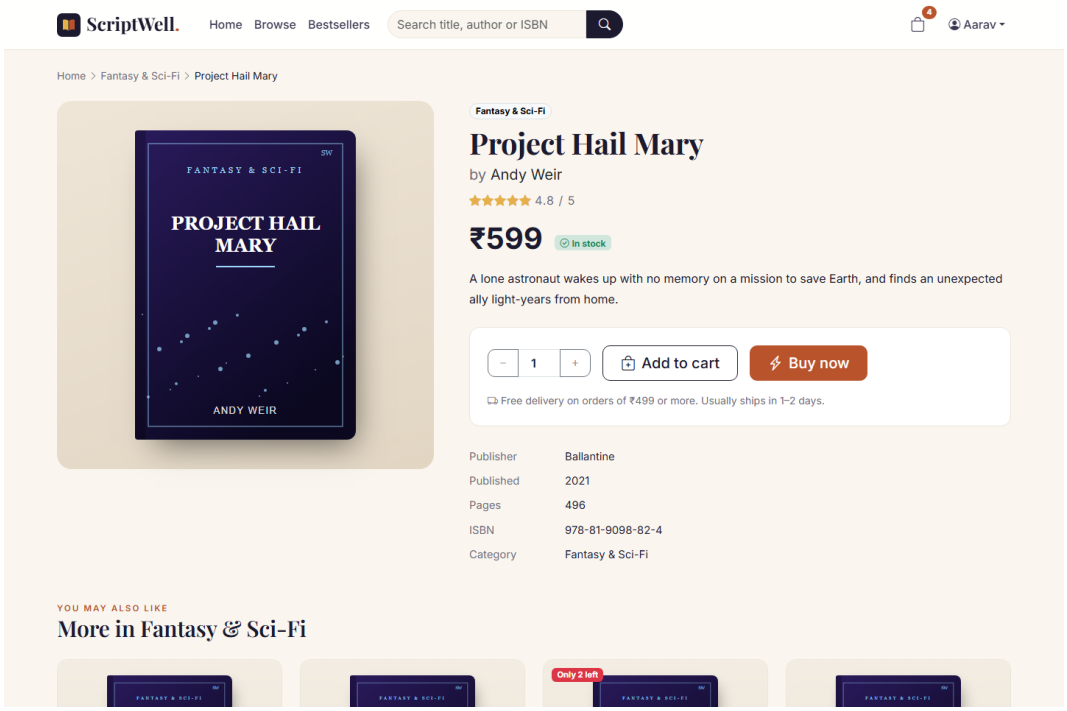


Figure 4.2: Book details page with Add to cart and Buy Now

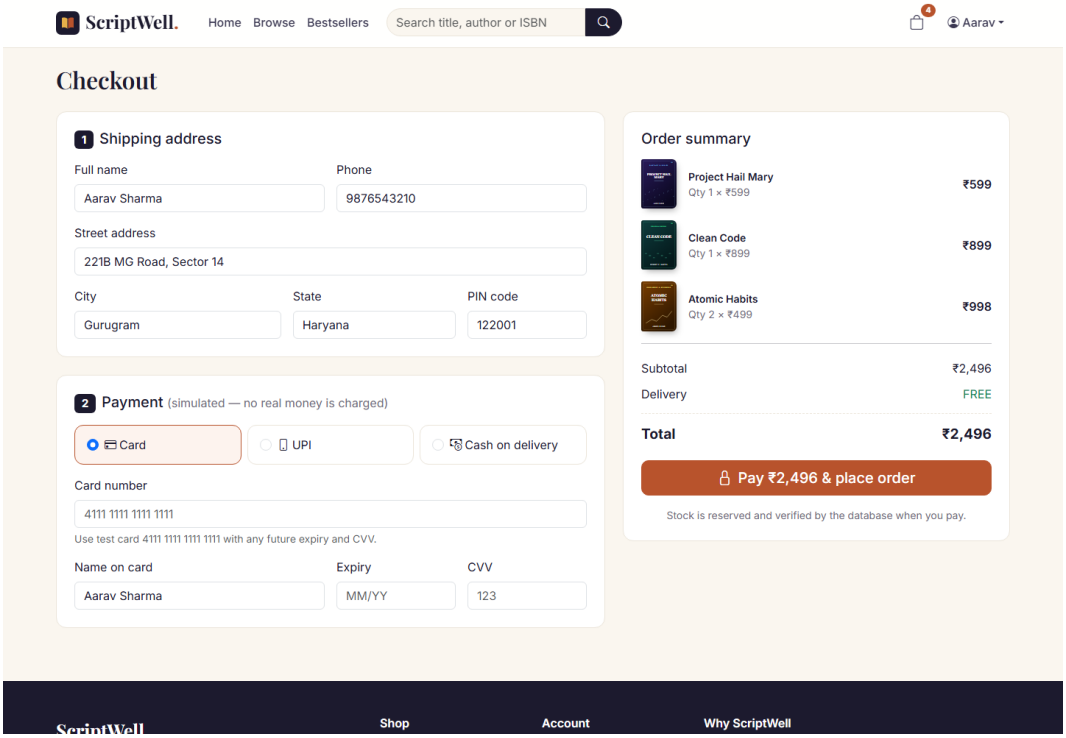


Figure 4.3: Checkout with simulated Card, UPI and Cash-on-delivery payment

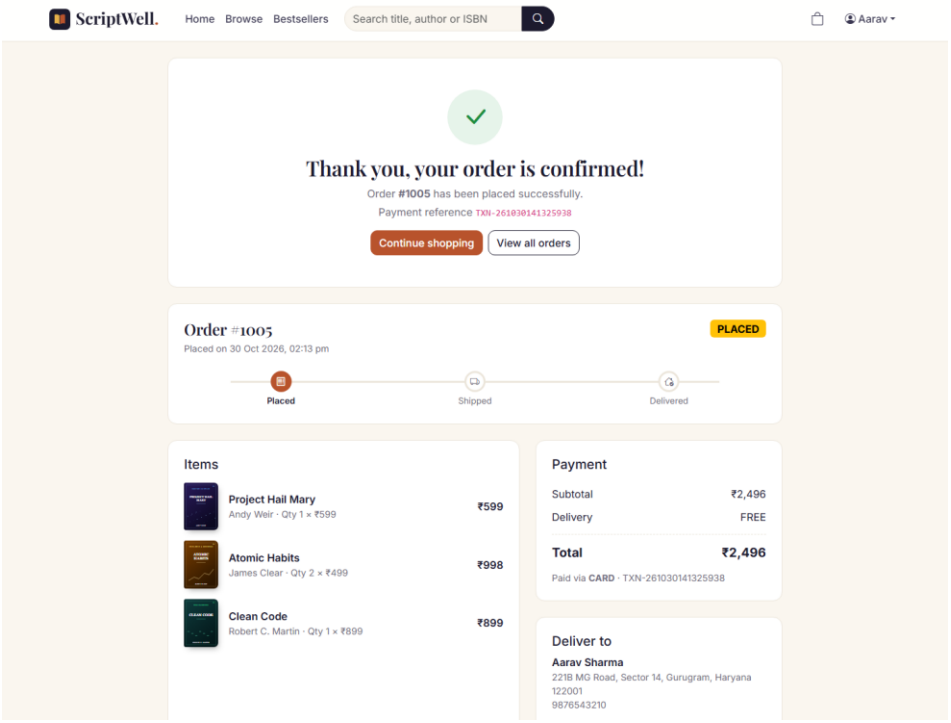


Figure 4.4: Order confirmation page with payment reference and status timeline

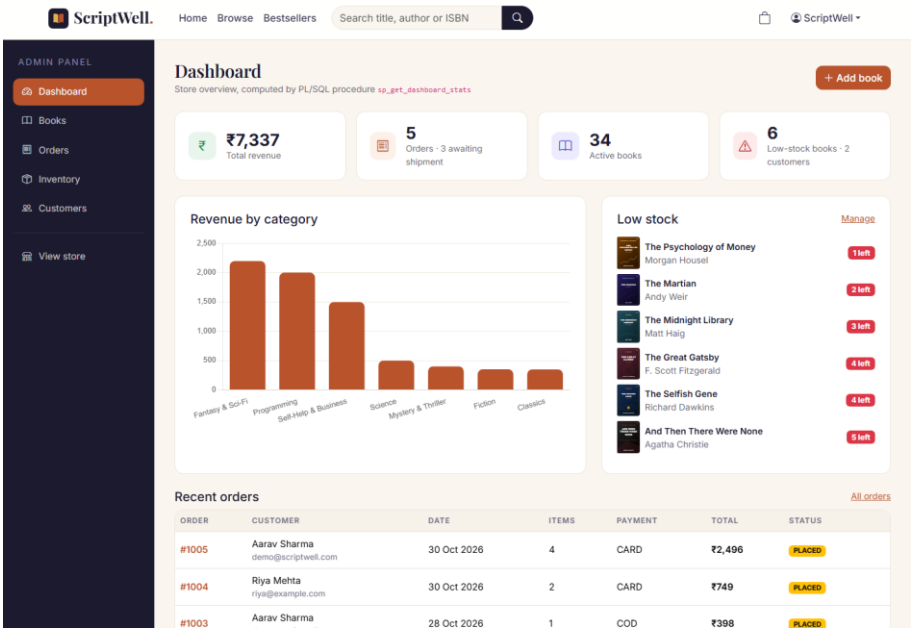


Figure 4.5: Administrator dashboard with statistics and sales chart

4.4 Limitations and Future Scope

- Integrate a real payment gateway (e.g. Razorpay or Stripe) in test mode with webhook confirmation.
- Send e-mail or SMS notifications when orders are placed or shipped.
- Add reviews, wish lists and recommendations, and image upload for book covers.
- Use a JNDI DataSource for connection pooling, and expose a REST API (JAX-RS) for a mobile app.

4.5 Conclusion

ScriptWell meets all of its objectives. The Servlet and JSP application follows a clean MVC structure, keeps user state with sessions, protects pages with filters and talks to Oracle only through `CallableStatement`. Bootstrap and JavaScript give it a modern, responsive and interactive interface. The project covers the main topics of Advanced Internet Programming in one working application that can be fully explained and demonstrated in a viva.

4.6 Pushing the Project to GitHub

The complete Eclipse project, SQL scripts and reports are published at **https://github.com/lovishv1718/scriptwell** using these commands:

```
git init -b main
git add .
git commit -m "ScriptWell online book store (Servlets, JSP, JDBC, Oracle PL/SQL)"
git remote add origin https://github.com/lovishv1718/scriptwell.git
git push -u origin main
```

REFERENCES-

[1] Eclipse Foundation, *Jakarta Servlet Specification 6.0* and *Jakarta Server Pages 3.1*, jakarta.ee.

[2] The Apache Software Foundation, *Apache Tomcat 10.1 Documentation*, tomcat.apache.org.

[3] Oracle Corporation, *Oracle Database JDBC Developer's Guide, 23ai*, docs.oracle.com.

[4] M. Hall and L. Brown, *Core Servlets and JavaServer Pages*, 2nd ed., Prentice Hall.

[5] Bootstrap Team, *Bootstrap 5.3 Documentation*, getbootstrap.com.

[6] Mozilla, *MDN Web Docs: Fetch API, HTML and CSS*, developer.mozilla.org.

[7] ScriptWell source code repository, https://github.com/lovishv1718/scriptwell.